"Choose Your Own" Project

Maria Sotnikova

2026-11-06

Executive Summary

For my "Choose Your Own" Project, I looked for data in the Kaggle repository as recommended by the assignment instructions. I found a data set called Adult Census Income that was uploaded to Kaggle by UCI Machine Learning and contains Census Bureau data from 1994. The variables included in the data set are:

- age
- workclass
- fnlwgt
- education
- education.num
- marital.status
- occupation
- relationship
- race
- sex
- capital.gain
- capital.loss
- hours.per.week
- native.country
- income

The goal of my project is to use all these variables to predict corresponding income. To do this, I imported, cleaned, and visualized the Adult Census Income from Kaggle. Then, I created a training set and testing set with a 80/20 split. I selected this split because it provides a substantial amount of data for training while reserving a separate portion for evaluating predictive performance. I used that data to train and test the K-Nearest Neighbors (KNN) algorithm and the Random Forest algorithm to predict whether a person's income exceeds $50K/yr. The KNN algorithm makes predictions based on nearby observations, while the Random Forest algorithm combines predictions from multiple decision trees. These two methods go beyond linear regression and have the potential to make more accurate predictions.

Analysis

First, let's take a look at the code used to perform data cleaning, data exploration, and visualization:

```
# Install and load the necessary packages
if (!require(tidyverse)) {
  install.packages("tidyverse")
  library(tidyverse)
}
```

```
## Loading required package: tidyverse
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
```

```
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ----------------------------------------- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
if (!require(caret)) {
  install.packages("caret")
  library(caret)
}
```

```
## Loading required package: caret
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```
if (!require(RKaggle)) {
  install.packages("RKaggle")
  library(RKaggle)
}
```

```
## Loading required package: RKaggle
```

```
if (!require(randomForest)) {
  install.packages("randomForest")
  library(randomForest)
}
```

```
## Loading required package: randomForest
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```
# Import directly into a tibble
census_data <- RKaggle::get_dataset("uciml/adult-census-income")
```

```
## Rows: 32561 Columns: 15
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (9): workclass, education, marital.status, occupation, relationship, rac...
## dbl (6): age, fnlwgt, education.num, capital.gain, capital.loss, hours.per.week
```

```
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

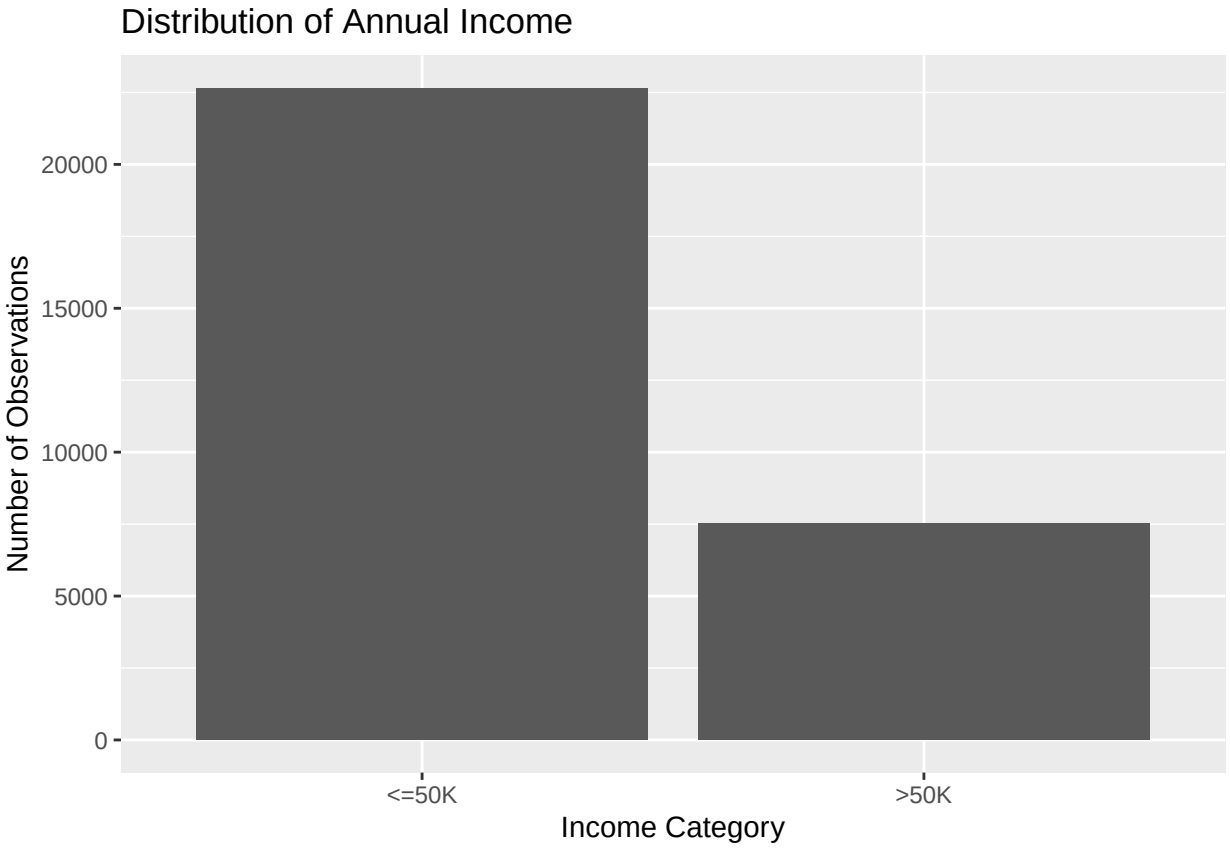
```
# Examine the data
print(head(census_data))
```

```
## # A tibble: 6 x 15
##     age workclass fnlwgt education    education.num marital.status occupation
##   <dbl> <chr>      <dbl> <chr>                <dbl> <chr>          <chr>
## 1    90 ?          77053 HS-grad                  9 Widowed        ?
## 2    82 Private   132870 HS-grad                  9 Widowed        Exec-manager~
## 3    66 ?         186061 Some-college            10 Widowed        ?
## 4    54 Private   140359 7th-8th                  4 Divorced       Machine-op-i~
## 5    41 Private   264663 Some-college            10 Separated      Prof-special~
## 6    34 Private   216864 HS-grad                  9 Divorced       Other-service
## # i 8 more variables: relationship <chr>, race <chr>, sex <chr>,
## #   capital.gain <dbl>, capital.loss <dbl>, hours.per.week <dbl>,
## #   native.country <chr>, income <chr>
```

```
# Remove any row that contains a "?", convert categorical variables to factors, and save to a new tibbl
census_data_clean <- census_data %>%
  mutate(across(where(is.character), str_trim)) %>%
  mutate(across(where(is.character), ~ na_if(.x, "?"))) %>%
  drop_na() %>%
  mutate(across(where(is.character), as.factor))

# Visualize the data
income_plot <- ggplot(census_data_clean, aes(x = income)) +
  geom_bar() +
  labs(
    title = "Distribution of Annual Income",
    x = "Income Category",
    y = "Number of Observations"
  )

print(income_plot)
```



Using this code, I noticed that there were missing values in the data, as well as categorical factors. So I cleaned the data by removing missing values and converting categorical variables to factors. Next I decided to split the data into a training set and a testing set in order to apply the K-Nearest Neighbors (KNN) algorithm. To optimize the performance of the K-Nearest Neighbors (KNN) algorithm, I performed hyperparameter tuning using three-fold cross-validation, with hyperparameter k set to three values: 5, 9, and 13. These parameters were chosen to reduce run time and produce the most accurate result. k = 13 produced the highest cross-validation accuracy.

```
# Create training set and testing set
set.seed(170)
train_index <- createDataPartition(y = census_data_clean$income, times = 1, p = 0.8, list = FALSE)
train_set <- census_data_clean[train_index, ]
test_set <- census_data_clean[-train_index, ]

# Separate predictors from the outcome
train_x <- train_set %>% select(-income)
test_x <- test_set %>% select(-income)

# Create dummy-variable transformation using training predictors only
dummy_model <- dummyVars(~ ., data = train_x, fullRank = FALSE)

# Apply transformation to training and testing sets
train_predictors <- predict(dummy_model, newdata = train_x)
test_predictors <- predict(dummy_model, newdata = test_x)

# Standardize training set
preprocess_model <- preProcess(train_predictors, method = c("center", "scale"))
```

```r
train_predictors_scaled <- predict(preprocess_model, train_predictors)

# Standardize testing set using the same parameters
test_predictors_scaled <- predict(preprocess_model, test_predictors)

# Store training set and testing set outcomes
train_income <- as.factor(train_set$income)
test_income <- factor(test_set$income, levels = levels(train_income))

# Define cross-validation
train_control <- trainControl(method = "cv", number = 3)

# Train KNN model
set.seed(170)
knn_model <- train(
  x = train_predictors_scaled,
  y = train_income,
  method = "knn",
  trControl = train_control,
  tuneGrid = data.frame(k = c( 5, 9, 13))
)

# Display results
print(knn_model)
```

```
## k-Nearest Neighbors
##
## 24131 samples
##   104 predictor
##     2 classes: '<=50K', '>50K'
##
## No pre-processing
## Resampling: Cross-Validated (3 fold)
## Summary of sample sizes: 16088, 16086, 16088
## Resampling results across tuning parameters:
##
##   k   Accuracy   Kappa
##    5  0.8193197  0.4946587
##    9  0.8220546  0.4972377
##   13  0.8244579  0.5015357
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 13.
```

```r
# Display optimal k
print(knn_model$bestTune)
```

```
##    k
## 3 13
```

```r
# Predict income using the testing dataset
y_hat_knn <- predict(knn_model, newdata = test_predictors_scaled)

# Generate confusion matrix
cm <- confusionMatrix(data = y_hat_knn, reference = test_income)
```

```r
# Display confusion matrix
print(cm)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K >50K
##      <=50K  4110  648
##      >50K    420  853
##
##                Accuracy : 0.8229
##                  95% CI : (0.813, 0.8325)
##     No Information Rate : 0.7511
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.501
##
##  Mcnemar's Test P-Value : 3.756e-12
##
##             Sensitivity : 0.9073
##             Specificity : 0.5683
##          Pos Pred Value : 0.8638
##          Neg Pred Value : 0.6701
##              Prevalence : 0.7511
##          Detection Rate : 0.6815
##    Detection Prevalence : 0.7889
##       Balanced Accuracy : 0.7378
##
##        'Positive' Class : <=50K
##
```

```r
# Display classification accuracy
print(cm$overall["Accuracy"])
```

```
##  Accuracy
## 0.8229149
```

The accuracy of the K-Nearest Neighbors (KNN) algorithm was found to be 0.823. Could the Random Forest algorithm perform better? I used the following code to find out. With the Random Forest algorithm, I also performed hyperparameter tuning using three-fold cross-validation, for accuracy and run time. I evaluated three values of mtry: 5, 10, and 20, and specified 100 trees. mtry = 10 produced the highest cross-validation accuracy.

```r
# Define parameters for Random Forest
set.seed(170)
control <- trainControl(method="cv", number = 3)
grid <- data.frame(mtry = c(5, 10, 20))

train_rf <-  train(train_predictors_scaled, train_income,
                   method = "rf",
                   ntree = 100,
                   trControl = control,
                   tuneGrid = grid)

# Display Random Forest cross-validation results
```

```r
print(train_rf)
```

```
## Random Forest
##
## 24131 samples
##   104 predictor
##     2 classes: '<=50K', '>50K'
##
## No pre-processing
## Resampling: Cross-Validated (3 fold)
## Summary of sample sizes: 16088, 16086, 16088
## Resampling results across tuning parameters:
##
##   mtry  Accuracy   Kappa
##    5    0.8503998  0.5571116
##   10    0.8622932  0.6064452
##   20    0.8619203  0.6089780
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 10.
```

```r
# Display optimal mtry
print(train_rf$bestTune)
```

```
##   mtry
## 2   10
```

```r
# Display detailed tuning results
print(train_rf$results)
```

```
##   mtry  Accuracy     Kappa  AccuracySD     KappaSD
## 1    5 0.8503998 0.5571116 0.003134115 0.006292688
## 2   10 0.8622932 0.6064452 0.001888087 0.004032506
## 3   20 0.8619203 0.6089780 0.001849861 0.004120829
```

```r
# Predict income using the testing dataset
y_hat_rf <- predict(train_rf, test_predictors_scaled)

# Generate confusion matrix
cm <- confusionMatrix(y_hat_rf, test_income)

# Display confusion matrix
print(cm)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K >50K
##      <=50K  4227  567
##      >50K    303  934
##
##                Accuracy : 0.8557
##                  95% CI : (0.8466, 0.8645)
##     No Information Rate : 0.7511
##     P-Value [Acc > NIR] : < 2.2e-16
##
```

```
##                   Kappa : 0.5901
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.9331
##             Specificity : 0.6223
##          Pos Pred Value : 0.8817
##          Neg Pred Value : 0.7551
##              Prevalence : 0.7511
##          Detection Rate : 0.7009
##    Detection Prevalence : 0.7949
##       Balanced Accuracy : 0.7777
##
##        'Positive' Class : <=50K
##
```

```
# Display classification accuracy
print(cm$overall["Accuracy"])
```

```
##  Accuracy
## 0.8557453
```

Results

```
# Store results
cm_knn <- confusionMatrix(y_hat_knn, test_income)
cm_rf <- confusionMatrix(y_hat_rf, test_income)

model_results <- tibble(
  Model = c("KNN", "Random Forest"),
  Accuracy = c(
    unname(cm_knn$overall["Accuracy"]),
    unname(cm_rf$overall["Accuracy"])
  )
)

# Display results
print(model_results)
```

```
## # A tibble: 2 x 2
##   Model         Accuracy
##   <chr>            <dbl>
## 1 KNN              0.823
## 2 Random Forest    0.856
```

The Random Forest algorithm performed better than the K-Nearest Neighbors (KNN) algorithm, achieving an accuracy of 0.856. Well done!

Conclusion

In summary, I used Adult Census Income data from Kaggle to train and test the K-Nearest Neighbors (KNN) algorithm and the Random Forest algorithm to predict whether a person's income exceeds $50K/yr. The Random Forest algorithm performed better than the K-Nearest Neighbors (KNN) algorithm. This methodology could potentially impact marketing strategies to better understand their audiences. It is however limited by the fnlwgt variable, shorthand for a set of weights on the Current Population Survey (CPS) file

that were developed by the data publisher. These weights could have biases. Future work could involve running these two algorithms again without fnlwgt as a predictor. Another limitation is that the distribution plot of annual income shows that the data set is imbalanced. Prediction accuracy alone may not adequately describe the effectiveness of the algorithms. Additional performance measures could be looked at in the future.

References

- Adult Census Income data from Kaggle: https://www.kaggle.com/datasets/uciml/adult-census-income
- Introduction to Data Science: Data Analysis and Prediction Algorithms with R by Rafael A. Irizarry, Chapter 32 - Machine learning in practice: https://rafalab.dfci.harvard.edu/dsbook/machine-learning-in-practice.html#machine-learning-in-practice
- ChatGPT: https://chatgpt.com/